

AULA 1

Introdução ao Framework e Arquitetura

Disciplina: CK0207 — Desenvolvimento de Software para Web

Professor: Dr. Fernando Antônio Mota Trinta

Universidade Federal do Ceará (UFC)

Projeto do curso: `nest-api-curso`

Autor: Samuel Sales Furtado

Objetivos de Aprendizagem

- Entender por que o NestJS foi criado
- Identificar problemas comuns do Express em aplicações grandes
- Compreender a arquitetura do Nest
- Entender o papel de Modules, Controllers e Services
- Compreender Injeção de Dependências
- Criar um projeto utilizando a Nest CLI

1. Motivação para o NestJS

O NestJS é, de propósito, um framework **opinitivo** — ou seja, ele tem uma opinião forte sobre como o código deve ser organizado, inspirada fortemente no Angular. Ele oferece injeção de dependências nativa, uma estrutura clara de módulos, controllers e services desde o primeiro dia, e roda por cima do próprio Express (ou, alternativamente, do Fastify). Isso significa que nada do ecossistema Node é perdido — apenas se ganha uma camada de organização por cima dele.

O ganho prático dessa opinião arquitetural, além da organização em si, é duplo: **testabilidade** e **capacidade de escalar o projeto com mais gente trabalhando ao mesmo tempo**. Quando todo o time segue o mesmo padrão de módulos, controllers e services, fica muito mais fácil entender o código de quem chegou depois, e mais fácil testar cada peça isoladamente.

2. Limitações do Express em Projetos Grandes

Para entender por que essa "opinião forte" do Nest é valiosa, vale olhar o outro lado: o que acontece na ausência dela. O Express é extremamente minimalista — oferece roteamento (um jeito de mapear uma URL para uma função) e middlewares (funções que rodam antes ou depois da rota), e nada além disso. Ele não define onde colocar a lógica de negócio, não define como organizar pastas, e não tem nenhum sistema de injeção de dependência embutido.

Isso é adequado para protótipos pequenos. À medida que o projeto cresce, essa ausência de estrutura **pode** virar um problema real — mas não *necessariamente* vira. Se o time tiver disciplina e definir bem as convenções desde o início, é possível manter um projeto Express grande organizado. O problema é que essa disciplina depende inteiramente do time, e nada no framework a impõe.

Na prática, o cenário mais comum quando essa disciplina falta é: cada desenvolvedor organiza o código à sua maneira, a injeção de dependência é feita manualmente (literalmente `new Servico()` espalhado pelo código), e não existe uma linha clara entre "código de servidor HTTP" e "regra de negócio". O resultado, quando isso foge do controle, são controllers gigantescos, lógica de negócio misturada com lógica de requisição/resposta, e um projeto cada vez mais difícil de testar e escalar.

3. Arquitetura do NestJS

Toda requisição numa aplicação NestJS segue um fluxo básico:

```
Cliente → Controller (HTTP) → Service (regra de negócio) → Banco / API externa → Resposta
```

Regra de ouro: o controller nunca fala diretamente com o banco de dados — ele sempre passa pelo service. Essa separação é o que permite, mais adiante no curso, trocar a forma como os dados são persistidos sem tocar no controller, ou testar a lógica de negócio isoladamente, sem precisar simular uma requisição HTTP inteira.

3.1 Controllers e Services

Controllers são responsáveis apenas por receber a requisição HTTP e devolver uma resposta. Eles não devem conter lógica de negócio — a função do controller é orquestrar entrada e saída, delegando qualquer trabalho mais complexo para um service. Um

controller com muitas linhas de `if`, cálculo ou validação manual é um sinal de que essa lógica deveria estar em outro lugar.

Services, também chamados de *providers*, são classes simples marcadas com o decorator `@Injectable()`. É aqui que mora a regra de negócio de verdade: processamento de dados, acesso ao banco, comunicação com serviços externos. O Nest gerencia o ciclo de vida dessas classes: por padrão, cada service é instanciado uma única vez, no início da aplicação, e essa mesma instância é reutilizada em toda a aplicação — o padrão *singleton*.

3.2 Modules

Um **módulo** é uma classe decorada com `@Module()` que agrupa controllers e providers relacionados a um mesmo domínio — uma "caixinha" que empacota tudo que pertence, por exemplo, ao domínio de usuários.

```
@Module({
  controllers: [UsersController],
  providers: [UsersService],
  imports: [OtherModule],
  exports: [UsersService],
})
export class UsersModule {}
```

Propriedade	Papel
<code>controllers</code>	Controllers deste módulo
<code>providers</code>	Services instanciados por este módulo
<code>imports</code>	Outros módulos cujos providers exportados este módulo também usa
<code>exports</code>	O que este módulo disponibiliza para quem o importar

Toda aplicação NestJS tem pelo menos um módulo raiz — normalmente chamado de `AppModule` — e cresce a partir daí, criando módulos de funcionalidade: um `UsersModule`, um `AuthModule`, e assim por diante.

4. Injeção de Dependências (DI)

O Nest possui um container de inversão de controle (*IoC container*) embutido. Em vez de o próprio controller criar a instância do service de que precisa com `new UsersService()`, ele apenas **declara** essa dependência no construtor, e é o Nest quem cria e entrega a instância:

```
@Injectable()
export class UsersService {}

@Controller('users')
export class UsersController {
  constructor(private usersService: UsersService) {}
}
```

4.1 Os três passos por trás do registro

1. `@Injectable()` na classe `UsersService` diz ao Nest que essa classe pode ser gerenciada pelo container IoC.
2. O **Controller declara a dependência** no construtor — a chamada injeção via construtor, forma mais comum de fazer isso no Nest.
3. No **módulo**, `providers: [UsersService]` registra a classe no container, associando o token `UsersService` à própria classe, para que possa ser resolvida sempre que solicitada.

4.2 Por que isso importa na prática

- **Testabilidade:** é possível substituir um service real por uma versão simulada (*mock*) sem tocar em uma linha do controller.
- **Desacoplamento:** o controller não precisa saber *como* o service é construído, apenas que ele existe e qual é o seu tipo.

- **Ciclo de vida:** por padrão, cada provider é um singleton — instanciado uma vez no bootstrap e reutilizado até a aplicação encerrar. Existe a possibilidade de escopo por requisição, mas isso é um tópico avançado, fora do escopo desta aula.

5. Nest CLI e Estrutura de Projetos

A Nest CLI é a ferramenta usada durante todo o curso para gerar a estrutura de um projeto NestJS.

```
npm install -g @nestjs/cli

nest new nome-do-projeto
```

O comando mais usado no dia a dia é `nest generate`, ou simplesmente `nest g`:

```
nest generate module cats      # ou: nest g mo cats
nest generate controller cats  # ou: nest g co cats
nest generate service cats     # ou: nest g s cats
```

A convenção de organização adotada neste curso é por **domínio/feature**, em vez de agrupar todos os controllers numa pasta e todos os services em outra:

```
src/
  users/
    dto/
      create-user.dto.ts
    users.controller.ts
    users.service.ts
    users.module.ts
  app.module.ts
  main.ts
```

Cada domínio isolado facilita localizar código relacionado e reduz o acoplamento entre partes diferentes do sistema.

Laboratório — Passo a Passo Esta seção acompanha, na prática, o exemplo do fluxo Controller → Service → Module. ## Passo 1 — Criar o projeto

```
npm install -g @nestjs/cli
nest new aula-1-demo
cd aula-1-demo
npm run start:dev
```

Testar: abrir `http://localhost:3000` — deve aparecer `Hello World!` .

Passo 2 — Gerar módulo, controller e service (um comando de cada vez)

```
nest g mo cats
nest g co cats
nest g s cats
```

Testar: `ls src/cats` deve mostrar `cats.controller.ts`, `cats.service.ts` e `cats.module.ts`. Abrir `src/app.module.ts` e confirmar que o `CatsModule` já foi importado automaticamente.

Passo 3 — Editar o `cats.service.ts`

```
import { Injectable } from '@nestjs/common';

export interface Cat {
  id: number;
  name: string;
  age: number;
}

@Injectable()
export class CatsService {
  private cats: Cat[] = [{ id: 1, name: 'Mingau', age: 2 }];

  findAll(): Cat[] {
    return this.cats;
  }

  findOne(id: number): Cat | undefined {
    return this.cats.find((cat) => cat.id === id);
  }
}
```

Passo 4 — Editar o `cats.controller.ts`

```
import { Controller, Get, Param, ParseIntPipe } from '@nestjs/common';
import { CatsService } from './cats.service';

@Controller('cats')
export class CatsController {
  constructor(private readonly catsService: CatsService) {}

  @Get()
  findAll() {
    return this.catsService.findAll();
  }

  @Get('/:id')
  findOne(@Param('id', ParseIntPipe) id: number) {
    return this.catsService.findOne(id);
  }
}
```

Testar: `curl http://localhost:3000/cats` deve devolver `[{"id":1,"name":"Mingau","age":2}]` .

Passo 5 — Conferir o `cats.module.ts` gerado automaticamente

```
@Module({
  controllers: [CatsController],
  providers: [CatsService],
})
export class CatsModule {}
```

Nenhuma edição é necessária aqui — a CLI já criou e conectou tudo sozinha.

Verificação de Conformidade

- ✔ Papel de Controllers e Services — conferido em docs.nestjs.com/providers e docs.nestjs.com/controllers .
- ✔ `@Injectable()` , injeção via construtor, registro em `providers` — conferido em docs.nestjs.com/fundamentals/custom-providers .
- ✔ Providers como singleton por padrão — conferido em docs.nestjs.com/providers .
- ✔ `@Module()` com `controllers` , `providers` , `imports` , `exports` — conferido em docs.nestjs.com/modules .
- ✔ Comandos da CLI (`nest new` , `nest generate` e atalhos) — conferido em docs.nestjs.com/cli/usages .